OFFICIAL DOCUMENTATION

Project Planning & Management: Streakly

1.1 Project Proposal

Overview

Streakly is a mobile-first application designed to help users maintain consistency toward goals (like studying or programming) by converting long-term objectives and uploaded learning materials into actionable daily plans using a serverless AWS architecture.

Objectives

The core objective is to generate personalized study tasks, review sessions, and quizzes grounded in the user's actual material via Retrieval-Augmented Generation (RAG) rather than generic prompts. Additionally, it aims to motivate users through a gamification engine (streaks, XP, badges), social accountability challenges, and real mobile push notifications.

Scope (MVP)

The initial scope includes user authentication, goal creation, AI-generated multi-day plans, private S3 document uploads, Bedrock Knowledge Base integration, deterministic streak tracking, basic gamification (XP/badges), scheduled push notifications via Amazon SNS, and progress dashboards.

1.2 Project Plan (Timeline & Milestones)

Based on the implementation roadmap, the project is divided into structured milestones:

Milestone	Phases	Deliverables
M1: Foundation & API	Phase 1, 2, 3	Setup of the Flutter application, Git repository, and AWS environment (Cognito, API Gateway, Lambda, DynamoDB) to handle core authentication and Goal CRUD operations.
M2: AI & Core Workflow	Phase 4, 5	Integration of the AI planner (Gemini/Bedrock), JSON schema validation, task retrieval, and the implementation of the deterministic streak engine.
M3: Gamification & SNS	Phase 6, 7	Development of the server-side XP ledger, badge rules, Flutter messaging token registration, and Amazon SNS push notification delivery.
M4: RAG Integration	Phase 8, 9	Configuration of private S3 uploads, Bedrock Knowledge Base indexing, and the generation of grounded learning materials with source metadata.

1.3 Task Assignment & Roles

- **Frontend (Flutter) Developer:** Responsible for the mobile app interface, device push token registration via Firebase Messaging, state management, and handling controlled presigned S3 upload flows.
- **Backend (AWS Serverless) Engineer:** Manages the API Gateway and Lambda functions, designs the DynamoDB single-table schema, and enforces domain rules for streaks and gamification server-side to prevent client manipulation.
- **AI/RAG Engineer:** Configures Amazon Bedrock Knowledge Bases, manages the AI orchestration layer, crafts precision prompts, and ensures the generation models return strictly validated structured JSON.
- **Cloud/DevOps Engineer:** Configures EventBridge Schedulers for reminders, handles CloudWatch observability (logs, metrics, alarms), writes automated tests, and ensures least-privilege IAM roles for all AWS services.

1.4 Risk Assessment & Mitigation Plan

Risk Identified	Impact	Mitigation Strategy
AI Hallucinations & Missing Context	HIGH	If relevant source material cannot be retrieved from the Knowledge Base, the system will return a controlled state (Error 422: Insufficient source context) rather than pretending missing material was found.
Client-Side Data Manipulation	HIGH	Gamification state (XP, badges, streaks, challenges) is intentionally persisted server-side. The backend never trusts client-submitted scores.
Duplicate Rewards / State Conflict	MEDIUM	Task completion and gamification events are designed to be idempotent. Repeated requests will return a 409 Conflict error and cannot duplicate XP or streaks.
Security & Credential Leaks	CRITICAL	No AWS, Gemini, or Bedrock credentials will be stored in the Flutter source code. Access is strictly scoped via Cognito and IAM least-privilege roles.

1.5 KPIs (Key Performance Indicators)

- **Grounding Quality Rate:** The percentage of AI-generated quizzes and plans that successfully utilize retrieved document context without falling back to generic responses or triggering insufficient context errors.
- **Notification Delivery Success:** The reliability rate of the EventBridge Scheduler triggering the Reminder Lambda and successfully publishing push notifications to FCM/APNs via Amazon SNS.
- **System Observability & Uptime:** Tracking backend performance, API rate limits, and internal errors (500/503 statuses) using CloudWatch metrics and alarms.